[illegible]

```
STATE.player_disabled = False
    STATE.equipped_weapon = "chair_leg"

    try:
        audio.play("chair_break", volume=0.8)
    except Exception:
        pass

def _update_chair_break_cutscene_state():
    """Advance the Mission 3 chair-break cinematic clock.

    Returns True while the cinematic is active, allowing game_update() to
    pause normal combat/player simulation while the cutscene hands control
    back to the breakout.
    """
    anim_t = getattr(STATE, "mission3_chair_break_anim", -1)
    if anim_t < 0:
        return False

    if anim_t >= _CB_GRAB:
        _grant_chair_leg_from_cutscene()

    if anim_t >= _CB_END:
        _grant_chair_leg_from_cutscene()
        STATE.mission3_chair_break_anim = -1
        STATE.request_chair_break_cutscene = False
        _set_objective("Sharpen the chair leg at the drain. Sebbow holds the door.")
        return True

    STATE.mission3_chair_break_anim = anim_t + 1
    return True

def _update_stage(keys):
    """Stage dispatcher.

    STAGE_INTERROGATION (Intro 3): advance the dialogue once open. Line and
    written beats auto-advance after a reading-time timeout; choice beats
    block until the player picks. When the script runs off the end, the
    breach has already fired — flip to STAGE_BREAKOUT and open the cell.

    STAGE_BREAKOUT (Intro 4): stealth-first escape. The player must reach
    the cell-block exit tile (south of the security door) to advance to
    STAGE_COMMS. Guards start unaware; striking them or making noise will
    alert the others via enemy_hear_shot. Picking up a guard's dropped
    sidearm lets the player engage at range (and escalates any nearby
    guards who hear the gunshot).

    STAGE_ARCHIVES (Intro 6): separate map, clear security, Mara dead-drop
    contact, steal the vaulted files, exit east toward HUD Order.

    Later stages (DEC_BREAK / BREAKOUT) remain stubs — only the objective
    text is set; full implementation lives in later stages.
    """
    global _dialogue_timer, _dialogue_index, _breach_flash, _light_flicker
    global _sebas_breach_t, _chair_struggle
    global _boots_approach_t
    stage = getattr(STATE, "mission3_stage", STAGE_INTERROGATION)

    if getattr(STATE, "m3_alarm_active", False):
        STATE.m3_alarm_timer = getattr(STATE, "m3_alarm_timer", 0) + 1

    if stage == STAGE_INTERROGATION:
        _set_objective("Survive the interrogation.")
        _update_interrogation()
        return

    # STAGE_BREAKOUT onward: keep ticking the breach animation clock (so
    # Sebbow's blob finishes and/or shoots in the doorway) and decay any
    # residual fx. The renderer reads these flags in every stage.
    if _sebas_breach_t >= 0 and _sebas_breach_t < 400:
        _sebas_breach_t += 1
        if _sebas_breach_t == 300:
            _detonate_cell_door()
        # At frame 240, the C4 brick is "seated" on the door hinge — play a
        # soft metallic click so the placement reads as a deliberate action,
        # not instant debris appearing.
        if _sebas_breach_t == 240 and "sebas_charge_placed" not in STATE.mission3_events:
            STATE.mission3_events.add("sebas_charge_placed")
            try:
                audio.play("rifle_handle", volume=0.45)
            except Exception:
                pass
    if _breach_flash > 0:
        _breach_flash -= 1
    if _light_flicker > 0:
        _light_flicker -= 1

    if stage == STAGE_BREAKOUT:
        _update_breakout_stage()
    elif stage == STAGE_GRAB_CUTSCENE:
        # Cutscene is playing (handled by scene_render.py). Once the request
        # flag is released, transition to STAGE_COMMS.
        if not getattr(STATE, "request_outside_cutscene", False):
            _start_comms_phase()
    elif stage == STAGE_COMMS:
        STATE.m3_comms_ambience_timer = getattr(
            STATE, "m3_comms_ambience_timer", 0) + 1
        _m3_check_window_comms_resupply()
        _update_window_mags()
        if getattr(STATE, "m3_relay_active", False):
            _set_objective("Tuning relay ...")
        elif not getattr(STATE, "m3_relay_tuned", False):
            if len(getattr(STATE, "enemies", [])) > 0:
                _set_objective(f"Clear Comms Wing: {len(STATE.enemies)} guard(s) left.")
            elif not getattr(STATE, "m3_comms_power_restored", False):
                _set_objective("Restore auxiliary power at Battery Backup.")
            elif not getattr(STATE, "m3_comms_auth_acquired", False):
                _set_objective("Copy relay authentication from the Security Booth.")
            else:
                _set_objective("Return to the relay and establish HQ contact.")
        else:
            _set_objective("Signal Castellan Beach Witness.")
            _maybe_trigger_archives_handoff()
    elif stage == STAGE_ARCHIVES:
        _m3_check_window_comms_resupply()
        _update_window_mags()
        _update_archives_stage()
    elif stage == STAGE_DEC_BREAK:
        # Phase 5 stub — cutscene after Dimitrov is secured and player exits east.
        _set_objective("Survive Belevasov's security sweep. (Phase 5 — pending)")
    elif stage == STAGE_BREAKOUT:
        # Phase 6 stub.
        _set_objective("Get the file onto the Kiev breakout uplink. (Phase 6 — pending)")

def _m3_exit_requirements():
    """Return (ready, reason) for leaving the cell-block area.

    The outside exit should not fire just because the player reaches the tile.
    It now requires the authored-cell beats: Sebbow has drawn attention away,
    guards are spawned/cleared, and the armory K-9 has been taken.
    """
    if not getattr(STATE, "m3_sebbow_lost_done", False):
        return False, "Wait for Sebbow to clear out."
    if not getattr(STATE, "mission3_guards_spawned", False):
        return False, "Secure the cell block first."
    guards_left = len(getattr(STATE, "enemies", []))
    if guards_left > 0:
        return False, f"{guards_left} guard(s) left."
    if not getattr(STATE, "m3_unlocked", False):
        return False, "Take the K-9 in the armory."
    return True, "Outside exit clear."

def _m3_check_window_comms_resupply():
    """Episodic Sinatrik resupply: Sebbow throws magazines when the player
    runs completely dry (Comms Wing + Archives).

    No cutscene: magazines fly through the world and land near the player.
    """
    if not getattr(STATE, "m3_window_ally_active", False):
        return
    if getattr(STATE, "m3_window_ally_hp", 0) <= 0:
        return
    if len(getattr(STATE, "m3_window_mags", [])) > 0:
        return  # already throwing

    weapon = getattr(STATE, "current_weapon", "pistol")
    if weapon == "m3_ak":
        empty = STATE.m3_ammo_in_mag <= 0 and STATE.m3_spare_mags <= 0
    elif weapon == "pistol":
        empty = STATE.ammo_in_mag <= 0 and STATE.spare_mags <= 0
    else:
        empty = False

    if empty:
        _trigger_window_mags_throw()

def _update_window_mags():
    """Update flying magazines: when they land near the player, grant ammo."""
    if not getattr(STATE, "m3_window_mags", None):
        return
    if not STATE.player:
        return

    px, py = STATE.player.rect.center
    remaining = []

    for mag in STATE.m3_window_mags:
        if mag.get("caught"):
            continue

        # Authored ballistic arc rather than frame-by-frame gravity. The old
        # version used very low horizontal speed with extremely strong gravity,
        # so the mags dropped beside Sebbow like heavy bricks. Interpolating
        # over a fixed flight time guarantees a clean throw toward Dimitrov while
        # the parabolic lift still makes it look physical.
        mag["age"] = mag.get("age", 0) + 1
        mag["life"] -= 1
        delay = mag.get("delay", 0)
        flight_age = max(0, mag["age"] - delay)
        duration = max(1, mag.get("duration", 38))

        # Let the catch point follow Dimitrov gently during flight. This avoids
        # misses when the player keeps moving without making the mag visibly
        # home or snap sideways.
        if flight_age > 0:
            follow = 0.075
            mag["target_x"] += (px - mag["target_x"]) * follow
            mag["target_y"] += (py - mag["target_y"]) * follow

        t = max(0.0, min(1.0, flight_age / duration))
        eased = t * t * (3.0 - 2.0 * t)  # smooth launch and catch
        old_x, old_y = mag["x"], mag["y"]
        base_x = mag["start_x"] + (mag["target_x"] - mag["start_x"]) * eased
        base_y = mag["start_y"] + (mag["target_y"] - mag["start_y"]) * eased
        arc = math.sin(math.pi * t) * mag.get("arc_height", 78.0)
        mag["x"] = base_x
        mag["y"] = base_y - arc
        mag["vx"] = mag["x"] - old_x
        mag["vy"] = mag["y"] - old_y
        mag["rot"] = mag.get("rot", 0.0) + 0.35

        # Resolve exactly at the catch point. Life expiry remains a safety net.
        if (flight_age >= duration) or mag["life"] <= 0:
            # Grant the mag
            if getattr(STATE, "current_weapon", "") == "m3_ak":
                STATE.m3_spare_mags = getattr(STATE, "m3_spare_mags", 0) + 1
            else:
                STATE.spare_mags = getattr(STATE, "spare_mags", 0) + 1

            # Floating text
            STATE.floating_texts.append([
                mag["x"], mag["y"] - 30,
                "+1 MAG",
                (200, 220, 160),
                55,
            ])

            try:
                audio.play("pickup", volume=0.55)
            except Exception:
                pass

            mag["caught"] = True
            continue

        remaining.append(mag)

    STATE.m3_window_mags = remaining

    # When all mags are caught, clear the list
    if not remaining:
        STATE.m3_window_mags = []

def _trigger_window_mags_throw():
    """Sebbow throws two magazines toward Dimitrov when the player is dry.

    Magazines now spawn directly from Sebbow's current position so the throw
    is clearly visible.
    """
    if not STATE.player:
        return
    if len(getattr(STATE, "m3_window_mags", [])) > 0:
        return

    px, py = STATE.player.rect.center

    # Use Sebbow's actual position if he's active as an ally
    if getattr(STATE, "m3_window_ally_active", False):
        window_x = getattr(STATE, "m3_window_ally_x", px - 180)
        window_y = getattr(STATE, "m3_window_ally_y", py - 80)
    else:
        window_x = px - 180
        window_y = py - 80

    # Two magazines with different trajectories (clear arc)
    mags = []
    for i, (ox, oy) in enumerate([(-18, -10), (16, -6)]):
        target_x = px + random.randint(-20, 20)
        target_y = py + random.randint(-20, 15)

        # Fixed-time ballistic throw. The second mag leaves a few frames later
        # and takes a slightly different/lower arc so the pair is readable.
        start_x = window_x + ox
        start_y = window_y + oy
        duration = 38 + i * 5
        mag = {
            "x": start_x,
            "y": start_y,
            "target_x": target_x,
            "target_y": target_y,
            "life": 180,
            "duration": duration,
            "delay": i * 8,
            "arc_height": 90.0 - i * 18.0,
            "vx": (target_x - start_x) / duration,
            "vy": (target_y - start_y) / duration,
            "rot": random.uniform(-0.4, 0.4),
            "age": 0,
            "caught": False,
            "start_x": start_x,
            "start_y": start_y,
        }
        mags.append(mag)

    STATE.m3_window_mags = mags

    # Small particle burst at throw origin
    try:
        effects.burst(window_x, window_y, (255, 200, 80), count=6, speed_min=0.8, life_min=10, life_max=20)
    except Exception:
        pass

    # Radio line from Sebbow (stage-aware, rotating).
    STATE.m3_window_mags_thrown_count = getattr(STATE, "m3_window_mags_thrown_count", 0) + 1
    stage = getattr(STATE, "mission3_stage", "")
    n = STATE.m3_window_mags_thrown_count
    if stage == STAGE_ARCHIVES:
        lines = [
            "SEBBOW: Dry? Catch — don't make me regret this!",
            "SEBBOW: Mag! Keep pushing the stacks!",
            "SEBBOW: Last spare. Make them count!",
        ]
    else:
        lines = [
            "SEBBOW: Catch! Catch!",
            "SEBBOW: Mag up — move!",
            "SEBBOW: Stay active!",
        ]
    line = lines[min(n - 1, len(lines) - 1)]
    try:
        from scene.ui import trigger_radio_chatter
        trigger_radio_chatter(line, color=COL_SEBBOW, duration=180)
    except Exception:
        pass

    try:
        audio.play("rifle_handle", volume=0.5)
    except Exception:
        pass

def _update_breakout_stage():
    _update_window_escape_route()

    # Guards enter the simulation only after the scripted breach/chair-leg
    # sequence is over and the player has actually entered the cell doorway.
    if (not getattr(STATE, "mission3_doorway_breached", False)
            and getattr(STATE, "mission3_weapon", "none") in ("shiv", "sidearm", "m3_ak")):
        _spawn_m3_guards_once()

    if not getattr(STATE, "mission3_breakout_announced", False):
        STATE.mission3_breakout_announced = True
        _radio("SEBBOW: They're awake now. South hall — move.",
               color=COL_SEBBOW, duration=200, key="breakout_announced")

    weapon = getattr(STATE, "mission3_weapon", "none")
    has_sidearm = getattr(STATE, "mission3_sidearm_acquired", False)
    guards_left = len(STATE.enemies)
    # Once every cell-block guard is down, remember it so the window-door
    # marker + arrow can show on the minimap regardless of later state.
    if getattr(STATE, "mission3_guards_spawned", False) and guards_left == 0:
        STATE.m3_guards_all_killed = True

    exit_ready, exit_reason = _m3_exit_requirements()
    if weapon == "chair_leg":
        _set_objective("Craft shiv at marked drain.")
    elif not getattr(STATE, "m3_sebbow_lost_done", False):
        _set_objective("Wait for Sebbow to clear out.")
    elif not getattr(STATE, "mission3_guards_spawned", False):
        _set_objective("Secure the cell block.")
    elif guards_left > 0:
        _set_objective(f"Clear Sections: {guards_left} guard(s) left.")
    elif not getattr(STATE, "m3_unlocked", False):
        _set_objective("Armory: Take the K-9 SMG.")
    elif exit_ready:
        _set_objective("Reach the outside exit.")
    else:
        _set_objective(exit_reason)

    if STATE.player is not None:
        exit_pos = (_EXIT_TILE_COL * TILE_SIZE + TILE_SIZE // 2,
                    _EXIT_TILE_ROW * TILE_SIZE + TILE_SIZE // 2)
        if _near(exit_pos, radius=TILE_SIZE * 0.6):
            exit_ready, exit_reason = _m3_exit_requirements()
            if exit_ready:
                STATE.mission3_stage = STAGE_GRAB_CUTSCENE
                STATE.mission_phase = "m3_outside_transition"
                STATE.mission3_transition_kind = "outside"
                STATE.request_outside_transition_cutscene = True
            else:
                _set_objective(exit_reason)
                wait_t = getattr(STATE, "m3_exit_block_text_timer", 0)
                if wait_t <= 0:
                    STATE.m3_exit_block_text_timer = 75
                    STATE.floating_texts.append([
                        exit_pos[0], exit_pos[1] - 36,
                        exit_reason,
                        (240, 170, 140), 70,
                    ])
        if getattr(STATE, "m3_exit_block_text_timer", 0) > 0:
            STATE.m3_exit_block_text_timer -= 1

def _start_comms_phase():
    """Intro 4 → Intro 5 transition: flip STAGE_BREAKOUT → STAGE_COMMS.

    This is the Phase 4 win. The cell-block exit fires, the breakout is
    marked cleared on STATE (so later stages can read it for narrative
    callbacks), and the objective updates to the Phase 5 stub text. The
    actual Sinatra-to-contact cutscene + comms relay points to Intro 5.
    """
    if getattr(STATE, "mission3_stage", None) == STAGE_COMMS:
        return  # idempotent

    _stop_m3_alarm()

    _build_comms_world()
    STATE.mission3_stage = STAGE_COMMS
    STATE.mission_phase = "m3_comms"
    STATE.mission3_cell_block_cleared = True
    # Comms checkpoint: reaching the Comms Wing is a respawn point AND a full
    # heal, so dying later in the mission doesn't punish the whole breakout.
    STATE.mission3_comms_checkpoint_reached = True
    if STATE.player is not None:
        STATE.player.hp = STATE.player.max_hp
    STATE.mission3_transition_kind = ""
    _set_objective("Reach the relay terminal. Re-establish HQ contact.")
    # Radio cue: a brief radio-static burst signals the transition into
    # the next phase's beats. Silently no-ops in headless test harnesses.
    try:
        audio.play("radio_static", volume=0.5)
    except Exception:
        pass

def _maybe_trigger_archives_handoff():
    """If the player reaches the Comms Wing exit after the
    relay is tuned, flip into STAGE_ARCHIVES."""
    if getattr(STATE, "mission3_stage", None) != STAGE_COMMS:
        return
    if not getattr(STATE, "m3_relay_tuned", False):
        return
    if STATE.player is None:
        return
    exit_pos = (
        _ARCHIVES_DOOR_COL * TILE_SIZE + TILE_SIZE // 2,
        _ARCHIVES_DOOR_ROW * TILE_SIZE + TILE_SIZE // 2,
    )
    if _near(exit_pos, radius=TILE_SIZE * 0.85):
        _start_archives_phase()

def _start_archives_phase():
    """Comms → Archives transition: load the Archives map and Phase 6 state."""
    if getattr(STATE, "mission3_stage", None) == STAGE_ARCHIVES:
        return  # idempotent

    _stop_m3_alarm()
    _build_archives_world()
    STATE.mission3_stage = STAGE_ARCHIVES
    STATE.mission_phase = "m3_archives"
    STATE.mission3_archives_checkpoint_reached = True
    if STATE.player is not None:
        STATE.player.hp = STATE.player.max_hp
    STATE.mission3_transition_kind = ""
    _set_objective("With Sebbow — clear the Archives, then find Mara's dead-drop (NW).")
    try:
        audio.play("radio_static", volume=0.55)
        audio.play("terminal_glitch", volume=0.40)
    except Exception:
        pass
    _radio(
        "SEBBOW: I'm with you. Stacks west, vault northeast. Don't dry the mag — I'll toss you one.",
        color=COL_SEBBOW, duration=240, key="archives_entered_sebbow",
    )
    _radio(
        "HANDLER: Mara's dead-drop is in the northwest clerk office. Clear a path.",
        color=COL_HANDLER, duration=240, key="archives_entered",
    )

def _mara_terminal_pos():
    return (
        _MARA_TERMINAL_COL * TILE_SIZE + TILE_SIZE // 2,
        _MARA_TERMINAL_ROW * TILE_SIZE + TILE_SIZE // 2,
    )

def _vaulted_file_pos():
    return (
        _VAULTED_FILE_COL * TILE_SIZE + TILE_SIZE // 2,
        _VAULTED_FILE_ROW * TILE_SIZE + TILE_SIZE // 2,
    )

def _archives_exit_pos():
    return (
        _ARCHIVES_EXIT_COL * TILE_SIZE + TILE_SIZE // 2,
        _ARCHIVES_EXIT_ROW * TILE_SIZE + TILE_SIZE // 2,
    )

def _archives_nav_target():
    """World position the player should head to next, or None."""
    if getattr(STATE, "mission3_stage", None) not in (STAGE_ARCHIVES, STAGE_DEC_BREAK):
        return None
    if getattr(STATE, "m3_mara_contact_active", False):
        return None
    # Only guide once the wing is clear of Nightwatch hostiles.
    if len(_archives_nightwatch_hostiles()) > 0:
        return None

    if getattr(STATE, "m3_vaulted_acquired", False):
        return _archives_exit_pos()
    if not getattr(STATE, "m3_mara_contacted", False):
        return _mara_terminal_pos()
    # Vault unlocked — guide to the file pedestal.
    return _vaulted_file_pos()

def _draw_archives_nav_arrow(cam_x, cam_y):
    """Floating arrow above Dimitrov pointing toward the next objective.

    Hides when the player is close enough that the interact prompt takes over,
    so the arrow never sits on top of menus/plaques/prompts.
    """
    if not STATE.player or STATE.player_dead:
        return
    target = _archives_nav_target()
    if target is None:
        return
    px, py = STATE.player.rect.centerx, STATE.player.rect.centery
    tx, ty = target
    dx, dy = tx - px, ty - py
    dist = math.hypot(dx, dy)
    # Interact range owns the last stretch — hide arrow to avoid clutter.
    if dist < 100:
        return
    # Inside the vaulted vault room while holding the floor arrow (room is small).
    pc, pr = px // TILE_SIZE, py // TILE_SIZE
    if target == _vaulted_file_pos() and 36 <= pc <= 44 and 4 <= pr <= 10:
        return

    angle = math.atan2(dy, dx)
    bob = math.sin(pygame.time.get_ticks() * 0.006) * 3
    sx = int(px - cam_x)
    # High above the sprite so it clears Sebbow labels / tutorials.
    sy = int(py - cam_y - 58 + bob)
    # Clamp so the arrow itself never draws under the top HUD.
    sy = max(64, min(HEIGHT - 100, sy))
    sx = max(20, min(WIDTH - 20, sx))

    col = (255, 200, 80)
    tip_len = 16
    tip = (sx + math.cos(angle) * tip_len, sy + math.sin(angle) * tip_len)
    left = (sx + math.cos(angle + 2.45) * 10, sy + math.sin(angle + 2.45) * 10)
    right = (sx + math.cos(angle - 2.45) * 10, sy + math.sin(angle - 2.45) * 10)
    pygame.draw.polygon(screen, (0, 0, 0),
                        [(tip[0] + 1, tip[1] + 1), (left[0] + 1, left[1] + 1),
                         (right[0] + 1, right[1] + 1)])
    pygame.draw.polygon(screen, col, [tip, left, right])

    # Distance plate ABOVE the arrow (not on the cork board / desk).
    meters = max(1, int(dist / 32))
    tag = font_terminal_small.render(f"{meters}m", True, col)
    tw, th = tag.get_width() + 8, tag.get_height() + 4
    tx0 = max(4, min(WIDTH - tw - 4, sx - tw // 2))
    ty0 = max(56, sy - th - 16)
    plate = pygame.Surface((tw, th), pygame.SRCALPHA)
    pygame.draw.rect(plate, (0, 0, 0, 160), plate.get_rect(), border_radius=4)
    pygame.draw.rect(plate, (*col, 180), plate.get_rect(), 1, border_radius=4)
    screen.blit(plate, (tx0, ty0))
    screen.blit(tag, (tx0 + 4, ty0 + 2))

def _archives_nightwatch_hostiles():
    """Enemies the player can fight right now (not trapped behind locked doors)."""
    locked_vault = not getattr(STATE, "m3_mara_contacted", False)
    locked_exit = not getattr(STATE, "m3_vaulted_acquired", False)
    out = []
    for e in getattr(STATE, "enemies", []):
        c = e.rect.centerx // TILE_SIZE
        r = e.rect.centery // TILE_SIZE
        # Outside sealed vaulted vault chamber.
```

```
# against the nearest wall, so Widow had no way to understand corners.
    follow_dist = 64
    target_x = px - math.cos(STATE.player.angle) * follow_dist
    target_y = py - math.sin(STATE.player.angle) * follow_dist
    dx = target_x - wx
    dy = target_y - wy
    dist = math.hypot(dx, dy)

    nav_walls = _ai_widow_navigation_walls(solid_walls)
    direct_clear = _ai_widow_segment_clear(
        wx, wy, target_x, target_y, nav_walls)
    goal_tile = (int(target_x // TILE_SIZE), int(target_y // TILE_SIZE))
    path = list(getattr(STATE, "ai_widow_path", []))
    path_timer = max(0, int(getattr(STATE, "ai_widow_path_timer", 0)) - 1)
    old_goal = getattr(STATE, "ai_widow_path_goal", None)

    # Replan when the player enters another tile, periodically while blocked,
    # or immediately after stuck-detection asks for a fresh route.
    if direct_clear:
        path = []
    elif path_timer <= 0 or old_goal != goal_tile or not path:
        path = _ai_widow_repath(wx, wy, target_x, target_y, nav_walls)
        path_timer = 18
        STATE.ai_widow_path_goal = goal_tile

    nav_x, nav_y = target_x, target_y
    if path:
        # Drop reached nodes.
        while path and math.hypot(path[0][0] - wx, path[0][1] - wy) < 18:
            path.pop(0)

        if path:
            # Path smoothing: take the furthest currently reachable waypoint,
            # avoiding robotic tile-center zig-zags in open corridors.
            furthest = 0
            for i, waypoint in enumerate(path[:6]):
                if _ai_widow_segment_clear(wx, wy, waypoint[0], waypoint[1], nav_walls):
                    furthest = i
                else:
                    break
            if furthest:
                path = path[furthest:]
            nav_x, nav_y = path[0]

    move_dx = nav_x - wx
    move_dy = nav_y - wy
    move_dist = math.hypot(move_dx, move_dy)
    if dist > 40 and move_dist > 1:
        step = min(3.05 * 60.0 * STATE.dt, move_dist)
        move_x = move_dx / move_dist * step
        move_y = move_dy / move_dist * step
        wx, wy = _ai_widow_move(wx, wy, move_x, move_y, solid_walls)

    # Stuck recovery is based on real displacement, not intended velocity.
    last_x, last_y = getattr(STATE, "ai_widow_last_pos", (wx, wy))
    moved = math.hypot(wx - last_x, wy - last_y)
    stuck = int(getattr(STATE, "ai_widow_stuck_timer", 0))
    if dist > 60 and moved < 0.35:
        stuck += 1
    else:
        stuck = 0
    if stuck >= 30:
        path = []
        path_timer = 0
        stuck = 0

    STATE.ai_widow_path = path
    STATE.ai_widow_path_timer = path_timer
    STATE.ai_widow_stuck_timer = stuck
    STATE.ai_widow_last_pos = (wx, wy)

    # Widow can open non-locked doors near her. Doors use the same target_open
    # / manual_timer logic as player doors, so they animate instead of popping.
    for door in getattr(STATE, "doors", []):
        if getattr(door, "locked", False):
            continue
        if getattr(door, "cutscene_disabled", False):
            continue
        if math.hypot(door.rect.centerx - wx, door.rect.centery - wy) <= TILE_SIZE * 1.35:
            door.target_open = True
            door.manual_timer = max(getattr(door, "manual_timer", 0), 24)

    # Find nearest visible guard.
    target = None
    best = 999999.0
    for e in getattr(STATE, "enemies", []):
        if e.hp <= 0:
            continue
        ex, ey = e.rect.centerx, e.rect.centery
        d = math.hypot(ex - wx, ey - wy)
        if d < best and d < 440 and not _wall_blocks_line(wx, wy, ex, ey, solid_walls):
            best = d
            target = e

    if target is not None:
        angle = math.atan2(target.rect.centery - wy, target.rect.centerx - wx)
        STATE.ai_widow_ally_angle = angle
        cd = max(0, int(getattr(STATE, "ai_widow_ally_cooldown", 0)) - 1)
        if cd <= 0:
            # Fire a real bullet using the same pipeline as the player instead
            # of directly subtracting HP. This gives Widow the same tracer,
            # wall collision, impact, casing, and timing behavior.
            spread = random.uniform(-0.035, 0.035)
            shot_a = angle + spread
            # Match the Jeffsud gun sprite: muzzle comes from chest height,
            # not from the sprite center/feet.
            muzzle_x = wx + math.cos(angle) * 30
            muzzle_y = wy - 7 + math.sin(angle) * 30
            STATE.bullets.append({
                "x": muzzle_x,
                "y": muzzle_y,
                "prev_x": muzzle_x,
                "prev_y": muzzle_y,
                "dx": math.cos(shot_a) * 38,
                "dy": math.sin(shot_a) * 38,
                "life": 40,
                "owner": "player",
                "damage": 45,
                "widow": True,
            })
            try:
                STATE.casings.append(Casing(
                    wx + math.cos(angle) * 8 + math.cos(angle - math.pi / 2) * 10,
                    wy + math.sin(angle) * 8 + math.sin(angle - math.pi / 2) * 10,
                    angle - math.pi / 2,
                ))
            except Exception:
                pass
            target.alerted = True
            target.last_seen_pos = (px, py)
            STATE.ai_widow_ally_shots.append([muzzle_x, muzzle_y, target.rect.centerx, target.rect.centery, 5])
            STATE.particles.append(Particle(muzzle_x, muzzle_y, (255, 230, 90)))
            try:
                audio.play("gunshot_pistol", volume=0.55)
            except Exception:
                pass
            cd = 34
        STATE.ai_widow_ally_cooldown = cd
    else:
        if dist > 4:
            STATE.ai_widow_ally_angle = math.atan2(dy, dx)
        STATE.ai_widow_ally_cooldown = max(0, int(getattr(STATE, "ai_widow_ally_cooldown", 0)) - 1)

    # Update tracer lifetimes.
    shots = []
    for shot in getattr(STATE, "ai_widow_ally_shots", []):
        shot[4] -= 1
        if shot[4] > 0:
            shots.append(shot)
    STATE.ai_widow_ally_shots = shots
    STATE.ai_widow_ally_x = wx
    STATE.ai_widow_ally_y = wy


def _draw_widow_ally(cam_x, cam_y):
    if not getattr(STATE, "ai_widow_ally_active", False):
        return
    if getattr(STATE, "mission3_stage", None) not in (STAGE_COMMS, STAGE_DEFENSE):
        return
    # Brief muzzle/tracer flash for Widow's support shots. The actual projectiles
    # is drawn by the normal bullet system; this is just a small firing cue.
    for sx, sy, ex, ey, life in getattr(STATE, "ai_widow_ally_shots", []):
        ssx, ssy = sx - cam_x, sy - cam_y
        esx, esy = ex - cam_x, ey - cam_y
        pygame.draw.line(screen, (255, 230, 90), (ssx, ssy), (esx, esy), 1)
        pygame.draw.circle(screen, (255, 240, 160), (int(ssx), int(ssy)), max(1, life))
    wx = getattr(STATE, "ai_widow_ally_x", 0.0) - cam_x
    wy = getattr(STATE, "ai_widow_ally_y", 0.0) - cam_y
    angle = getattr(STATE, "ai_widow_ally_angle", 0.0)
    _draw_widow_in_game(screen, wx, wy, angle, shot_phase=0, charge_phase=0)

def game_update(keys):
    """Per-frame update. Returns (cam_x, cam_y) for scribe's renderer."""
    global _last_keys
    _last_keys = keys
    # NOTE: Tick i-frames: advance player's frame timer + sprite blink every frame.
    # Mission 3 is interrogation/crafting-focused, but the player can still
    # take damage during the escape sequence -- i-frames must apply uniformly
    # across the campaign so the player's damage-state expectations are
    # consistent.
    if STATE.player:
        STATE.player.update_iframes(STATE.dt)
    if STATE.gt_flash_timer > 0:
        STATE.gt_flash_timer -= 1

    if _update_black_badger_death():
        STATE._death_logic_ticks = 0
        effects.update_particles()
        effects.update_floating_texts()
        return _update_camera()

    if STATE.player and STATE.player.hp <= 0 and not STATE.player_dead:
        _stop_ai_alarm()
        import death_anim
        death_anim.trigger_death()
    if STATE.player_dead:
        import death_anim
        ticks = death_anim.update_death_anim()
        STATE._death_logic_ticks = ticks
    else:
        STATE._death_logic_ticks = 0

    if _update_chair_break_cutscene_state():
        effects.update_particles()
        effects.update_floating_texts()
        return _update_camera()

    _update_doors()
    solid_walls = _solid_walls()
    _update_player(keys, solid_walls)

    _update_discreet_melee(solid_walls)
    _update_weapon_timers()
    _update_widow_ally(solid_walls)
    if (getattr(STATE, "current_weapon", "pistol") == "kd_bag"
            and not getattr(STATE, "mission3_crafting_active", False)
            and not getattr(STATE, "player_dead", False)):
        try:
            if pygame.mouse.get_pressed()[0]:
                import weapons as _weapons
                _weapons.handle_shoot()
        except Exception:
            pass

    guards_active = _m3_guards_active()
    if guards_active:
        _update_m3_enemies(solid_walls)
        _update_m3_cameras(solid_walls)
    _update_bullets(solid_walls)
    _update_black_badger_remote()
    if getattr(STATE, "m3_black_badger_death_active", False):
        effects.update_particles()
        effects.update_floating_texts()
        return _update_camera()
    _update_m3_pickups()
    _update_smoke_ring()
    _update_m3_defenses()
    effects.update_particles()
    effects.update_floating_texts()
    _update_stage(keys)
    _update_crafting()
    return _update_camera()

# ----------------------------------------------------
# Mission 3 — enemy + pickup per-frame loops
# ----------------------------------------------------
def _m3_guards_active():
    """True when Mission 3 guards should run combat/pathfinding.

    Guards used to activate during the Widow breach/interrogation and pile up
    against Widow's cell while the player was still locked in scripted beats.
    Freeze AI until the actual breakout is playable, and keep them held back
    while the mandatory chair-leg crafting lock is active (Widow is holding
    the doorway in-fiction).
    """
    stage = getattr(STATE, "mission3_stage", STAGE_INTERROGATION)
    if stage in (STAGE_INTERROGATION, STAGE_WAIT_OUTSIDE, STAGE_DONE):
        return False
    if (stage == STAGE_BREAKOUT
            and getattr(STATE, "mission3_doorway_blocked", False)
            and getattr(STATE, "mission3_weapon", "none") == "chair_leg"):
        return False
    return True

def _m3_enemy_has_strict_los(enemy, px, py, solid_walls):
    """Mission 3 hard-line LOS: center-to-center only.

    The shared Enemy._can_see() samples several points around the player so AI
    does not forget targets as they corner-clip in open maps. In this tight
    prison layout that felt like guards were seeing through cell/room walls.
    Mission 3 uses stricter center-line visibility for detection and shooting.
    """
    return _m3_enemy_has_los_to_point(enemy, px, py, solid_walls)

def _m3_enemy_has_los_to_point(enemy, tx, ty, solid_walls):
    if not enemy:
        return False
    ex, ey = enemy.rect.centerx, enemy.rect.centery
    dist = math.hypot(tx - ex, ty - ey)
    aware = getattr(enemy, "alerted", False) or getattr(enemy, "combat_alert_timer", 0) > 0
    max_range = ENEMY_DETECT_RANGE * (1.6 if aware else 1.05)
    if dist > max_range:
        return False
    return not _wall_blocks_line(ex, ey, tx, ty, solid_walls)

def _m3_widow_target_pos():
    if not getattr(STATE, "ai_widow_ally_active", False):
        return None
    if getattr(STATE, "mission3_stage", None) not in (STAGE_COMMS, STAGE_DEFENSE):
        return None
    if getattr(STATE, "ai_widow_ally_hp", 0) <= 0:
        return None
    return (float(getattr(STATE, "ai_widow_ally_x", 0.0)),
            float(getattr(STATE, "ai_widow_ally_y", 0.0)))

def _m3_choose_enemy_target(enemy, player_x, player_y, solid_walls):
    """Pick whether this guard targets Wildcat or Widow.

    If both are visible, guards split their attention roughly 50/50 and keep the
    choice for a short window to avoid jitter. If only one target is visible,
    use that target. If neither is visible, fall back to last-known behavior.
    """
    player_visible = _m3_enemy_has_los_to_point(enemy, player_x, player_y, solid_walls)
    widow_pos = _m3_widow_target_pos()
    widow_visible = False
    if widow_pos is not None:
        widow_visible = _m3_enemy_has_los_to_point(enemy, widow_pos[0], widow_pos[1], solid_walls)

    # Keep current target briefly if still valid.
    timer = max(0, int(getattr(enemy, "m3_target_timer", 0)) - 1)
    current = getattr(enemy, "m3_target_kind", "player")
    if timer > 0:
        if current == "widow" and widow_visible:
            enemy.m3_target_timer = timer
            return "widow", widow_pos[0], widow_pos[1], True
        if current == "player" and player_visible:
            enemy.m3_target_timer = timer
            return "player", player_x, player_y, True

    if player_visible and widow_visible:
        # Stable-ish 50/50 split. Use random at retarget moments only.
        if random.random() < 0.5:
            enemy.m3_target_kind = "widow"
            enemy.m3_target_timer = random.randint(45, 75)
            return "widow", widow_pos[0], widow_pos[1], True
        enemy.m3_target_kind = "player"
        enemy.m3_target_timer = random.randint(45, 75)
        return "player", player_x, player_y, True
    if widow_visible:
        enemy.m3_target_kind = "widow"
        enemy.m3_target_timer = random.randint(30, 60)
        return "widow", widow_pos[0], widow_pos[1], True
    if player_visible:
        enemy.m3_target_kind = "player"
        enemy.m3_target_timer = random.randint(30, 60)
        return "player", player_x, player_y, True

    # No current LOS: continue toward last known target if alert/searching.
    last = getattr(enemy, "last_seen_pos", None)
    if last and (getattr(enemy, "alerted", False)
                 or getattr(enemy, "combat_alert_timer", 0) > 0
                 or getattr(enemy, "state", "") in ("CHASE", "ATTACK", "SEARCH")):
        return getattr(enemy, "m3_target_kind", "player"), last[0], last[1], False
    return "none", -99999, -99999, False

def _m3_enemy_update_target(enemy, px, py, strict_can_see):
    """Target passed into the shared Enemy.update().

    If strict LOS says the player is hidden, do not feed the real player
    position into Enemy.update(), because its looser multi-point visibility can
    reacquire through corners/walls. Alerted guards instead act on last-known
    position; unaware guards get an off-map dummy point so they keep patrolling.
    """
    if strict_can_see:
        return px, py
    last = getattr(enemy, "last_seen_pos", None)
    if last and (getattr(enemy, "alerted", False)
                 or getattr(enemy, "combat_alert_timer", 0) > 0
                 or getattr(enemy, "state", "") in ("CHASE", "ATTACK", "SEARCH")):
        return last
    return (-99999, -99999)

def _update_m3_enemies(solid_walls):
    if not STATE.player or STATE.player_dead:
        return
    px, py = STATE.player.rect.centerx, STATE.player.rect.centery

    for e in STATE.enemies[:]:
        if e.hp > 0:
            target_kind, tx, ty, can_see = _m3_choose_enemy_target(e, px, py, solid_walls)

            # Feed Enemy.update() the chosen target (Wildcat or Widow). If
            # neither is visible it receives last-known/dummy coordinates so
            # it cannot reacquire through walls.
            e.update(tx, ty, solid_walls, STATE.bullets, STATE.particles,
                     STATE.enemies, getattr(STATE, "fire_zones", []))

            # Re-evaluate LOS after movement.
            if target_kind == "widow":
                wpos = _m3_widow_target_pos()
                if wpos is not None:
                    tx, ty = wpos
                    can_see = _m3_enemy_has_los_to_point(e, tx, ty, solid_walls)
                else:
                    can_see = False
            elif target_kind == "player":
                tx, ty = px, py
                can_see = _m3_enemy_has_los_to_point(e, tx, ty, solid_walls)
            else:
                can_see = False

            dist = math.hypot(e.rect.centerx - tx, e.rect.centery - ty)

            if can_see:
                e.alerted = True
                e.combat_alert_timer = 300
                etype = getattr(e, "m3_type", "guard")
                attack_range = getattr(e, "m3_attack_range", ENEMY_ATTACK_RANGE)

                # Type identity beyond visuals:
                # - breachers rush close before firing
                # - marksmen hold longer sightlines and backpedal if rushed
                # - technicians alert the room once when they spot you
                if etype == "breacher" and dist > attack_range:
                    e.state = "CHASE"
                    e.last_seen_pos = (tx, ty)
                    e.speed = max(getattr(e, "speed", 1.0), 1.85)
                elif etype == "marksman" and dist < 150:
                    # Backpedal away from the current target while keeping aim.
                    away = math.atan2(e.rect.centery - ty, e.rect.centerx - tx)
                    if hasattr(e, "_apply_move"):
                        e._apply_move(math.cos(away) * 1.0, math.sin(away) * 1.0, solid_walls)
                    e.state = "ATTACK"
                    e.last_seen_pos = (tx, ty)
                elif etype == "technician" and not getattr(e, "m3_called_alert", False):
                    e.m3_called_alert = True
                    STATE.floating_texts.append([e.rect.centerx, e.rect.centery - 40, "CALLING IT IN", (90, 140, 190), 70])
                    for other in getattr(STATE, "enemies", []):
                        other.combat_alert_timer = max(getattr(other, "combat_alert_timer", 0), 180)
                        other.alerted = True
                    e.state = "ATTACK" if dist < attack_range else "CHASE"
                    e.last_seen_pos = (tx, ty)
                elif dist < attack_range:
                    e.state = "ATTACK"
                    e.last_seen_pos = (tx, ty)
                else:
                    e.state = "CHASE"
                    e.last_seen_pos = (tx, ty)
            elif e.state == "ATTACK":
                # Do not keep firing after LOS breaks.
                e.state = "CHASE" if getattr(e, "last_seen_pos", None) else "SEARCH"

            if e.state == "ATTACK" and can_see and e.fire_cooldown <= 0:
                spread = random.uniform(-getattr(e, "m3_spread", 0.06), getattr(e, "m3_spread", 0.06))
                shot_angle = math.atan2(ty - e.rect.centery, tx - e.rect.centerx)
                bx = e.rect.centerx + math.cos(shot_angle) * 30
                by = e.rect.centery + math.sin(shot_angle) * 30
                bullet_speed = getattr(e, "m3_bullet_speed", 30)
                STATE.bullets.append({
                    "x": bx, "y": by,
                    "prev_x": bx, "prev_y": by,
                    "dx": math.cos(shot_angle + spread) * bullet_speed,
                    "dy": math.sin(shot_angle + spread) * bullet_speed,
                    "life": 60 if getattr(e, "m3_type", "guard") == "marksman" else 36,
                    "owner": "enemy",
                    "damage": getattr(e, "m3_damage", ENEMY_DAMAGE),
                    "enemy_type": getattr(e, "m3_type", "guard"),
                })
                e.fire_cooldown = getattr(e, "m3_fire_cooldown", ENEMY_FIRE_COOLDOWN)
                try:
                    vol = 0.55 if getattr(e, "m3_type", "guard") == "marksman" else 0.38
                    audio.play("gunshot_pistol", volume=vol)
                except Exception:
                    pass

        if e.hp <= 0:
            STATE.enemies_killed += 1
            STATE.mission3_guards_killed += 1
            STATE.blood_decals.append((
                e.rect.centerx, e.rect.centery, 0.0, random.randint(14, 24)))
            effects.burst(e.rect.centerx, e.rect.centery, C_BLOOD, 30)
            effects.shake(4)

            # FIRST-PISTOL GUARANTEE: the first guard killed in Mission 3 while
            # the player still lacks a firearm ALWAYS drops a usable sidearm
            # (pistol) — 100% of the time — so the player can never get stuck
            # with only the improvised melee weapon. This uses a dedicated
            # one-shot flag (not the drop counter) so scripted/other drops
            # can't consume the guarantee and leave the first real guard
            # dropping only a random mag.
            label = getattr(e, "m3_label", "")
            sidearm_pickup_already_spawned = any(
                len(pu) >= 3 and pu[2] == "sidearm"
                for pu in getattr(STATE, "pickups", [])
            )
            first_pistol_pending = (
                not getattr(STATE, "mission3_first_pistol_dropped", False)
                and not getattr(STATE, "mission3_sidearm_acquired", False)
                and not getattr(STATE, "m3_unlocked", False)
                and not sidearm_pickup_already_spawned
            )
            if first_pistol_pending:
                STATE.mission3_first_pistol_dropped = True
                STATE.pickups.append([e.rect.centerx, e.rect.centery, "sidearm"])
                # Guarantee one emergency bandage with the first pistol so the
                # player is never sent into the breakout with zero healing.
                if getattr(STATE, "mission3_bandages_spawned", 0) < 1:
                    STATE.mission3_bandages_spawned += 1
                    STATE.pickups.append([
                        e.rect.centerx + random.randint(-8, 8),
                        e.rect.centery + random.randint(-8, 8),
                        "bandage"])
            else:
                # Drop economy: after the guaranteed first sidearm, guards only
                # sometimes drop usable mags. Elderly/unit guards have a slightly
                # better chance; extra mag drops are rare and capped so the
                # breakout doesn't turn into a loot pinata.
                mag_chance = 0.22 + (0.08 if label in ("guard_armory", "guard_exit_checkpoint") else 0.0)
                can_drop_extra_ammo = (
                    getattr(STATE, "mission3_sidearm_acquired", False)
                    or getattr(STATE, "m3_unlocked", False)
                )
                if (can_drop_extra_ammo
                        and getattr(STATE, "mission3_ammo_drops_spawned", 0) < 3
                        and random.random() < mag_chance):
                    STATE.mission3_ammo_drops_spawned += 1
                    STATE.pickups.append([e.rect.centerx, e.rect.centery, "sidearm"])

            # Bandages are scarce in M3 but now noticeably more reliable: up to
            # three emergency wraps per breakout route.
            if (getattr(STATE, "mission3_bandages_spawned", 0) < 3
                    and random.random() < 0.45):
                STATE.mission3_bandages_spawned += 1
                STATE.pickups.append([
                    e.rect.centerx + random.randint(-8, 8),
                    e.rect.centery + random.randint(-8, 8),
                    "bandage"])
            try:
                audio.play("bullet_impact_enemy", volume=0.45)
            except Exception:
                pass
            STATE.enemies.remove(e)

def _m3_black_badger_player_hit_count():
    return sum(1 for d in getattr(STATE, "mission3_detainees", [])
               if d.get("black_badger_hit_by_player"))

def _m3_spawn_black_badger_remote():
    """Spawn the Black Badger punishment entity.

    This is intentionally NOT a normal Enemy: it is unkillable, ignores normal
    hostile counts/drops, and owns a custom death animation.
    """
    if getattr(STATE, "m3_black_badger_triggered", False):
        return

    STATE.m3_black_badger_triggered = True
    STATE.m3_black_badger_remote_active = True
    STATE.m3_black_badger_intro_timer = 150
    STATE.m3_attack_active = True
    STATE.m3_attack_timer = 0

    px = STATE.player.rect.centerx if STATE.player else _EXIT_TILE_COL * TILE_SIZE
    py = STATE.player.rect.centery if STATE.player else _EXIT_TILE_ROW * TILE_SIZE

    # Spawn away from the player but inside the authored exit-block footprint.
    candidates = [
        (_EXIT_TILE_COL * TILE_SIZE + TILE_SIZE // 2, _EXIT_TILE_ROW * TILE_SIZE + TILE_SIZE // 2),
        (10 * TILE_SIZE + TILE_SIZE // 2, 24 * TILE_SIZE + TILE_SIZE // 2),
        (40 * TILE_SIZE + TILE_SIZE // 2, 17 * TILE_SIZE + TILE_SIZE // 2),
        (_DOOR_TILE_COL * TILE_SIZE + TILE_SIZE // 2, (_DOOR_TILE_ROW + 4) * TILE_SIZE + TILE_SIZE // 2),
    ]
    sx, sy = max(candidates, key=lambda p: math.hypot(p[0] - px, p[1] - py))

    STATE.m3_black_badger_remote = {
        "x": float(sx),
        "y": float(sy),
        "angle": math.atan2(py - sy, px - sx),
        "timer": 0,
        "lunge_timer": 0,
        "footstep": 0,
    }

    _set_objective("SURVIVE THE BLACK BADGER")
    STATE.floating_texts.append([
```

[illegible]

```
                                pygame.draw.line(scrim, (0, 0, 0, a), (0, t), (WIDTH, t))
    screen.blit(scrim, (0, y))


def _draw_dialogue_panel():
    """Render the minimal-horror Stories I interrogation interface.

    This intentionally avoids obvious UI furniture: no cards, no large panels,
    no boxed objectives.  Dialogue becomes sparse subtitle text over the void;
    choices become bare thoughts/responses, like things Bitten can force out
    while restrained.
    """
    global _dialogue_shake
    beat = _current_beat()
    if beat is None:
        return

    if _dialogue_shake > 0:
        _dialogue_shake = max(0.0, _dialogue_shake - 1.0)
    shake_offset = int(_dialogue_shake)

    btype = beat.get("type")
    speaker = beat.get("speaker", "")
    accent = beat.get("color", C_WHITE)
    ticks = pygame.time.get_ticks()
    cursor_visible = (ticks // 500) % 2 == 0

    # Oppressive focus layer: always slight; heavier when the player is forced
    # to choose.  This replaces the previous UI cards with mood.
    dim = pygame.Surface((WIDTH, HEIGHT), pygame.SRCALPHA)
    dim_alpha = 38 if btype != "choice" else 90
    dim.fill((0, 0, 0, dim_alpha))
    screen.blit(dim, (0, 0))
```

[illegible]

[illegible]